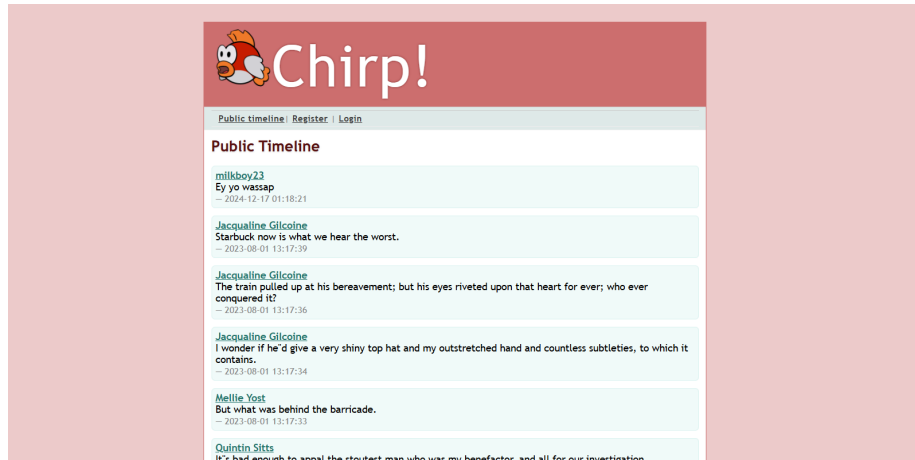


Chirp! Project Report



ITU BDSA 2024 Group 28

Analysis, Design and Software Architecture - BSANDSA1KU

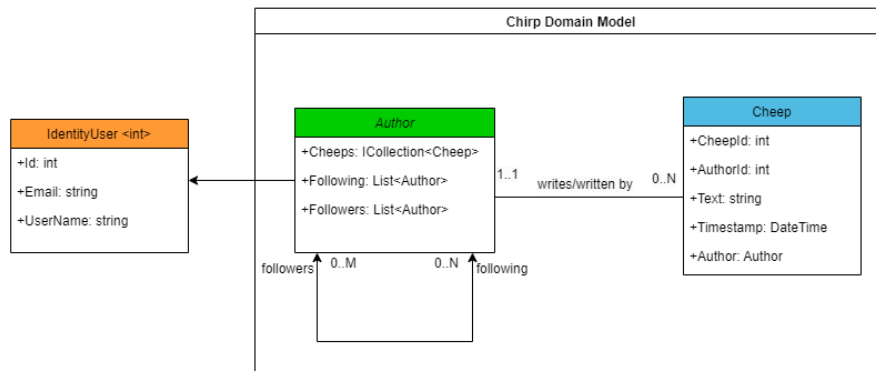
- Amira Maria Ayoub aayo@itu.dk
- Emma Andrea Ravn Krause-Kjær ekra@itu.dk
- Josefine Kamp Nielsen kajn@itu.dk
- Stine Helena Hallberg Sørensen sths@itu.dk
- Um-Kulsum Abusheva umab@itu.dk

Table of contents

- Design and Architecture
 - Domain model
 - Architecture - In the small
 - Architecture of deployed application
 - User activities
 - Sequence of functionality/calls through *Chirp!*
 - * Public timeline sequence
 - * Following/unfollowing sequence
 - * Forget me sequence
- Process
 - Build, test, release, and deployment
 - * Build and test workflow
 - * Release workflow
 - * Deployment workflow
 - Team Work
 - * Description of the group's workflow
 - How to make *Chirp!* work locally
 - * Using a release
 - For Windows
 - For MacOS X
 - * Git Cloning
 - How to run test suite locally
- Ethics
 - License
 - LLMs, ChatGPT, CoPilot, and others

Design and Architecture

Domain model

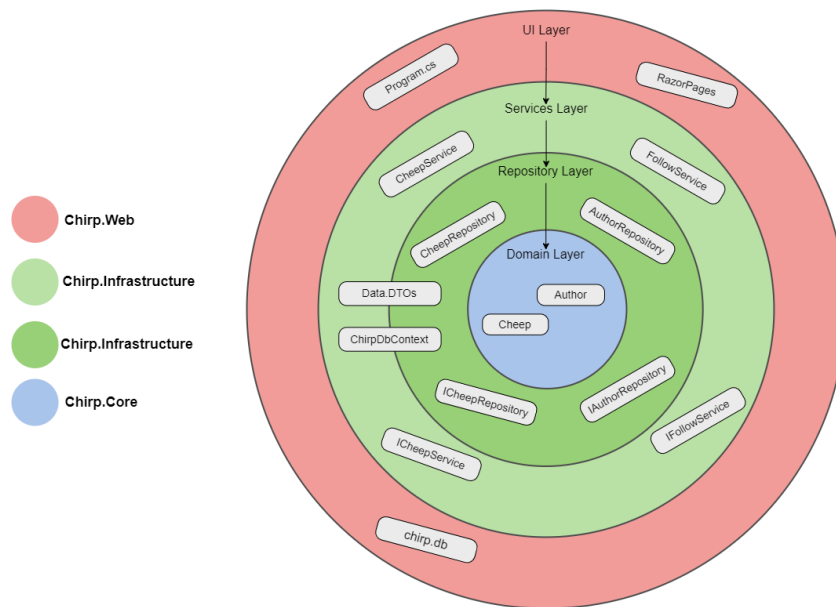


The Domain Model for the *Chirp!* application is implemented in the `Chirp.Core` package, which is the innermost layer in our Onion Model. This model consists of two main classes/entities: **Author** and **Cheep**.

The **Author** class represents a user of *Chirp!*. The class extends from the `IdentityUser` class from `AspNetCore.Identity`, allowing functionality such as authenticating a user. As seen in the above diagram, an **Author** automatically inherits an `Id` (`int`), an `Email` (`string`) and a `username` (`string`). Furthermore, an **Author** has a relation to **Cheep** by storing a `Collection` of cheeps the user has written. This ensures that every **Cheep** is written by only one **Author**, but an **Author** can write many **Cheeps**, making it a one-to-many relationship. Finally, the **Author** class stores two `Lists`, which contain the other **Authors** which the user is either following or followed by.

The **Cheep** class represents the structure for individual posts created by an authenticated user. It contains a unique identifier `CheepId` (`int`), a foreign key `AuthorId` (`int`) and an `Author` of type `Author`, associated with an existing user. It also contains the field `Text` (`string`), which is the content of the post with a maximum length of 160 characters. Lastly, it contains a `Timestamp` (`DateTime`), which is the registered time and date of when the cheep was posted.

Architecture — In the small



The Chirp Application attempts to follow The Onion Architecture, which ensures the separation of concerns and testability in our project. The architecture is implemented across three solutions:

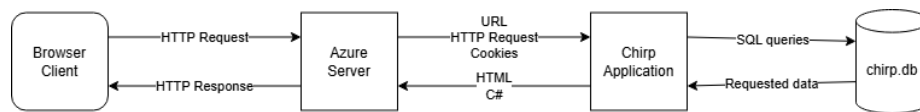
1. Chirp.Core (Domain Layer): As the innermost layer, this solution is responsible for defining the entities (Cheep, Author). This layer is seen in detail in the **Domain model** above. Being the core of the project makes it completely independent of external dependencies, only providing the foundation upon which all other layers build.

2. Chirp.Infrastructure (Repository and Services Layers): This solution has two important layers. The Repository Layer implements the methods of the entities, making them dependent on the Domain Layer. The Services Layer primarily interacts with the repositories and thereby the Domain Layer. Furthermore, the Infrastructure solution also consists of the ChirpDbContext, which acts as a bridge, integrating the Domain Model with the actual database (chrip.db) in the UI Layer.

3. Chirp.Web (UI Layer): The outermost layer of the Onion Architecture is implemented in the Chirp.Web solution and handles all user interaction through Razor Pages while interacting with the DTOs and Services. Located here is also the application's Program.cs file, which serves as the entry point for *Chirp!*. The Program.cs file is responsible for configuring and registering essential components,

such as the application's services, database context, authentication providers, and middleware pipeline.

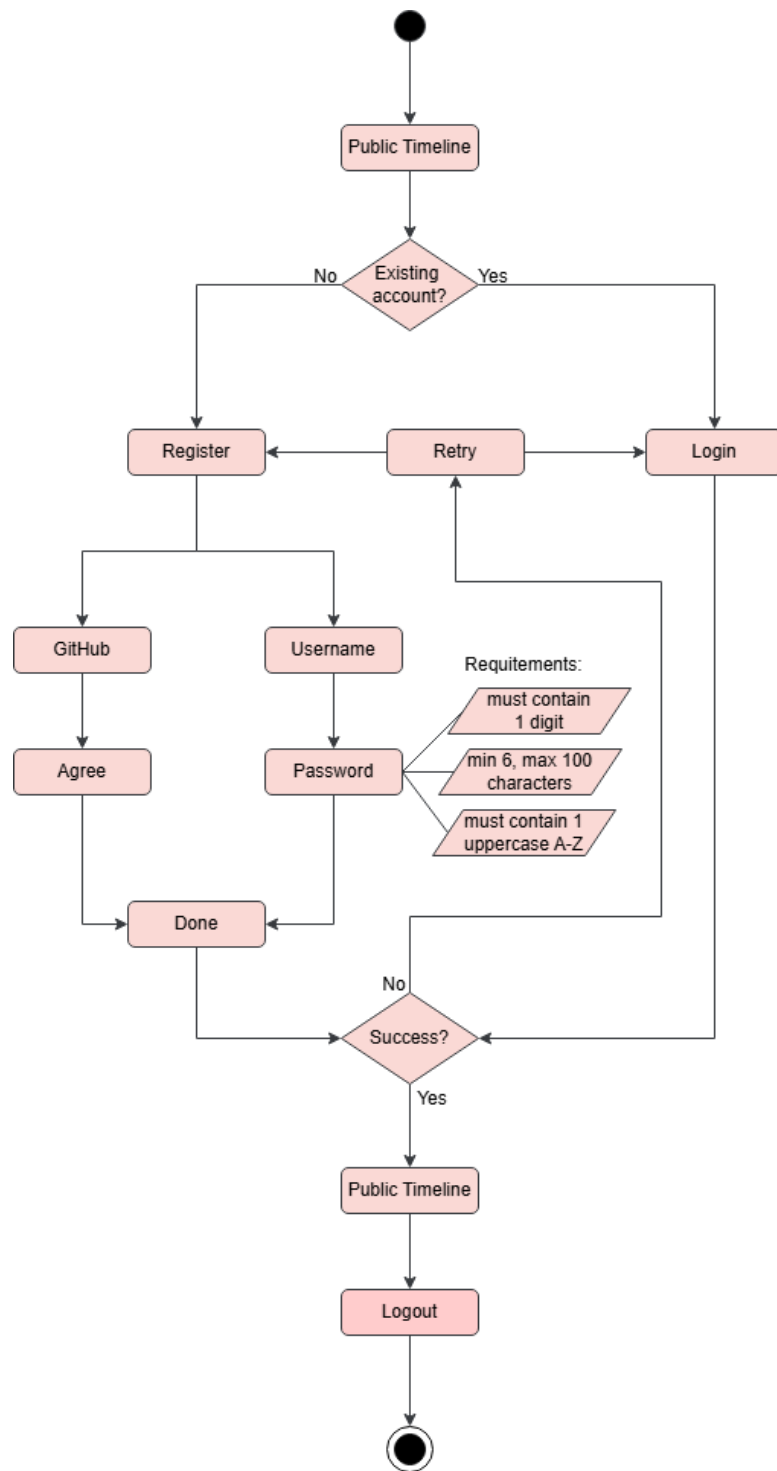
Architecture of deployed application



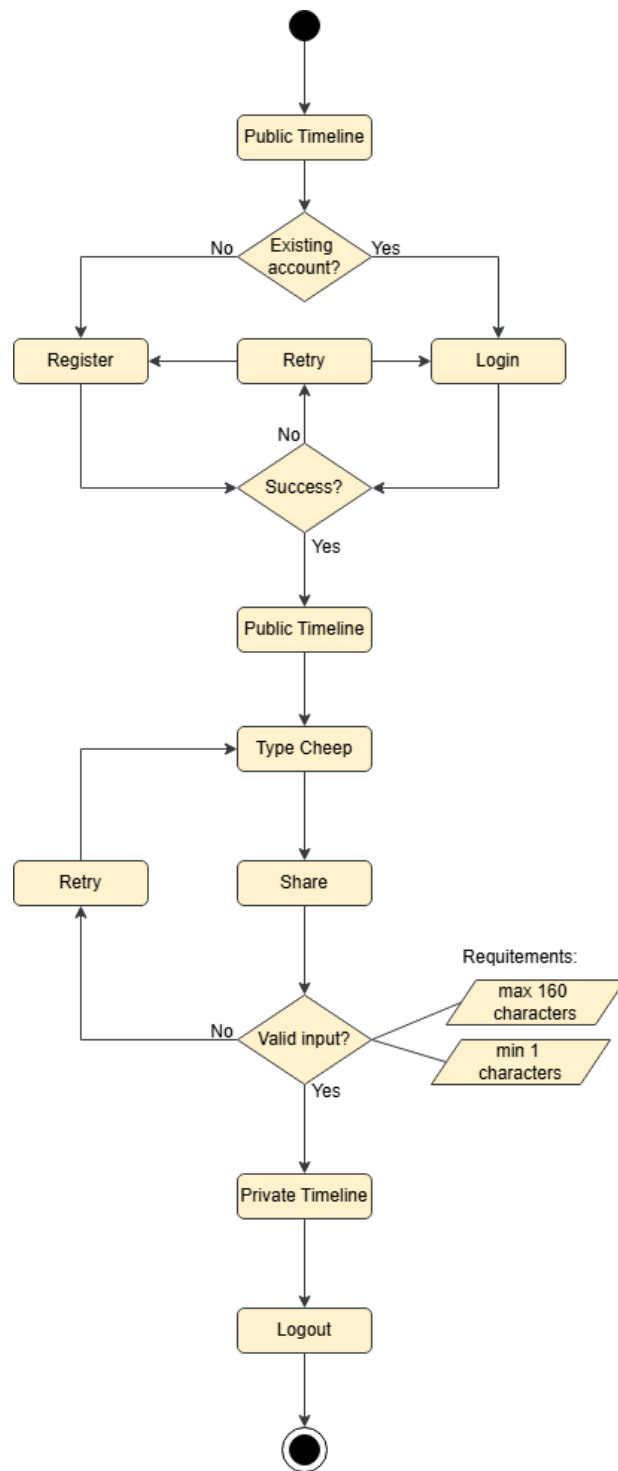
When the application gets deployed, the browser sends a request to the Azure server. Azure forwards the request to our application. The *Chirp!* application queries the database for the needed data. The database then returns the requested data, which the application turns into HTML and C# code. This results in the HTTP response that is visible to the users.

User activities

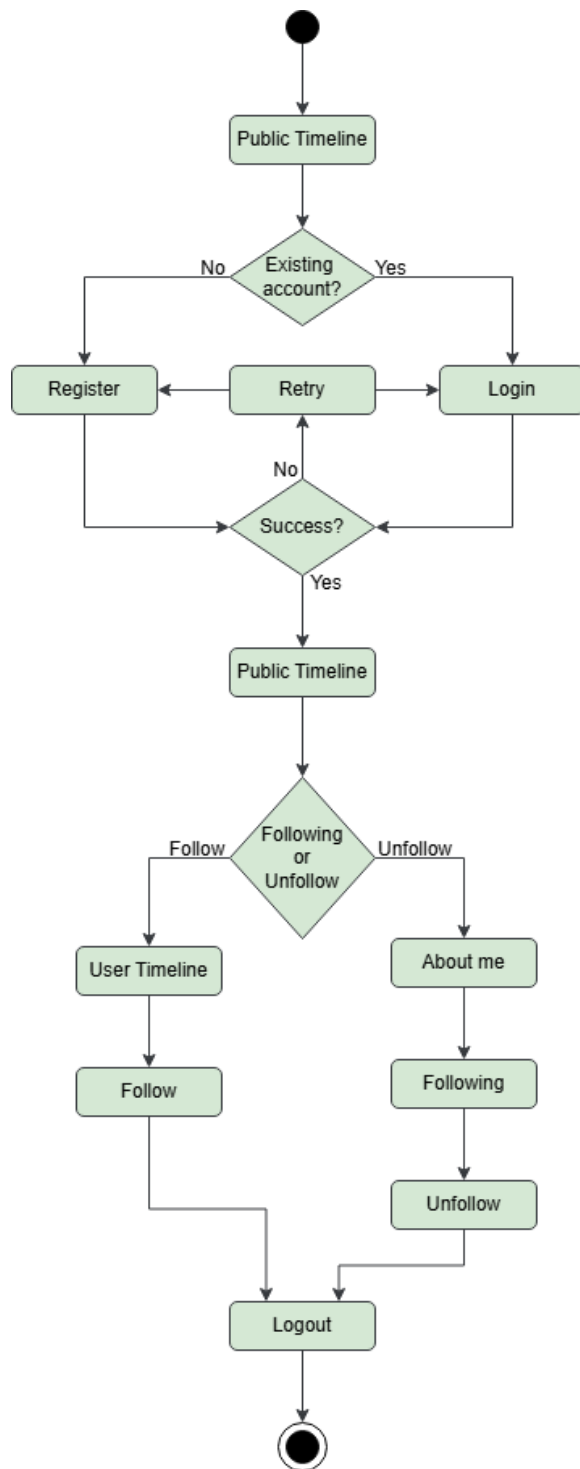
There are a number of actions a user can take on *Chirp!*. Below are diagrams to show the processes.



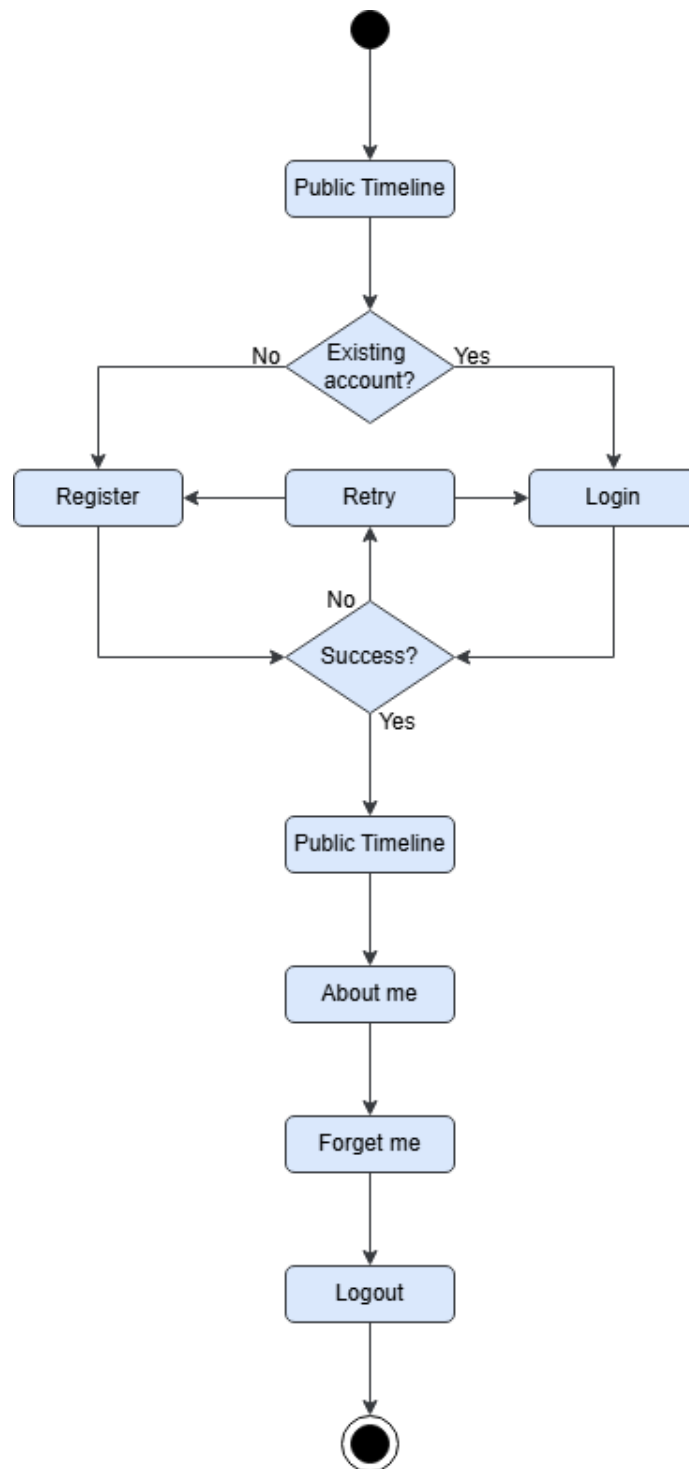
The diagram above shows the process of a user logging in.



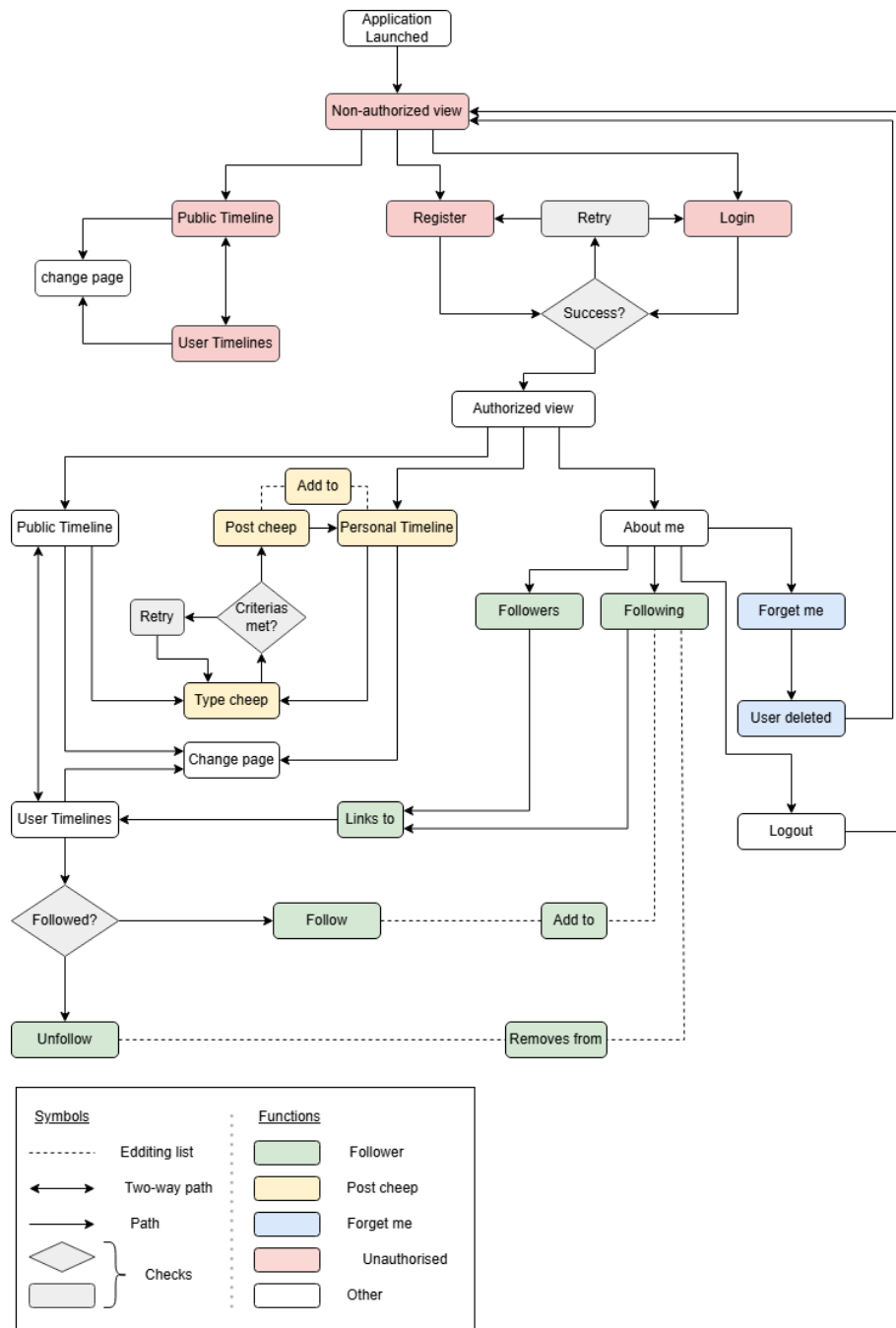
The diagram above shows the process of a user logging in and posting a cheep.



The diagram above shows the process of a user logging in and following another user.



The diagram above shows a user logging in to delete their account, by clicking the “Forget me” button on the User Profile page. This action results in the deletion of the user’s cheeps, along with being deleted from all its followers’ lists of following.

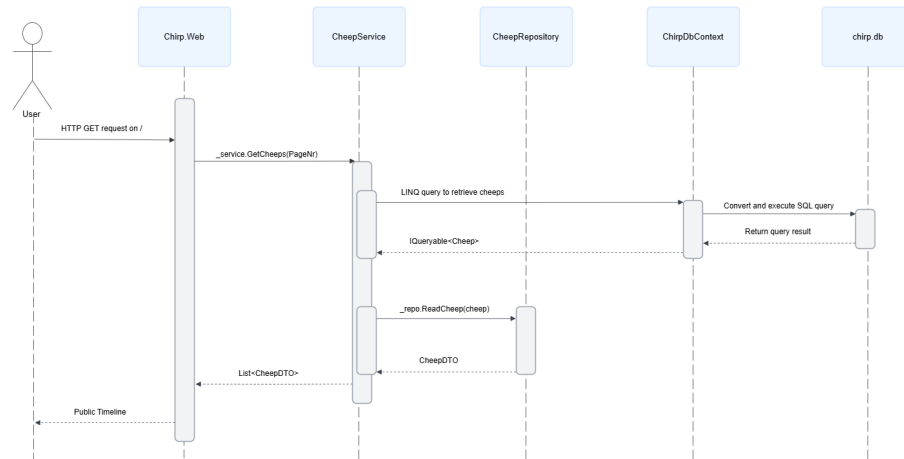


The diagram above shows a full diagram of the actions a user can take on the website. A user can end the application at any time by closing the window.

Sequence of functionality/calls through *Chirp!*

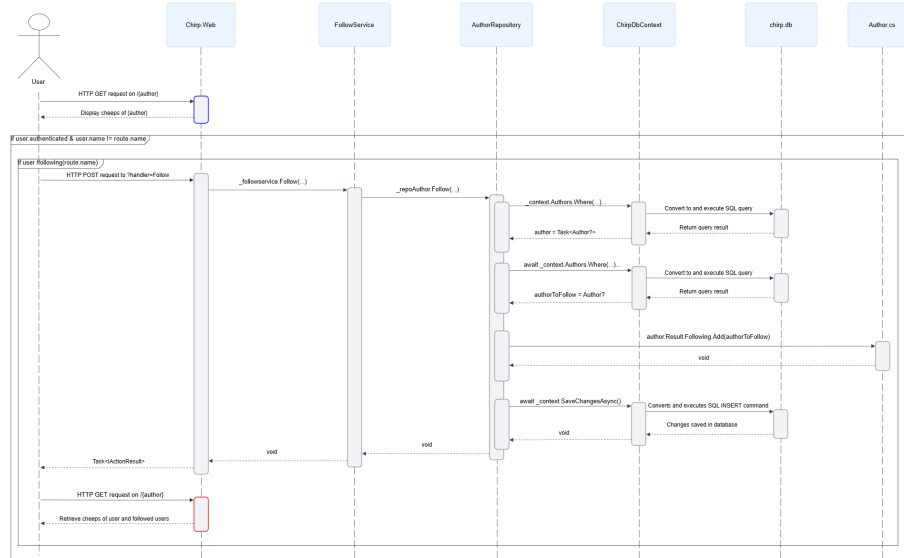
In the following section, a selection of the implemented functionality will be presented with the aid of sub-system sequence diagrams. The diagrams show the roles of the different components and programming languages in the project.

Accessing the Public Timeline



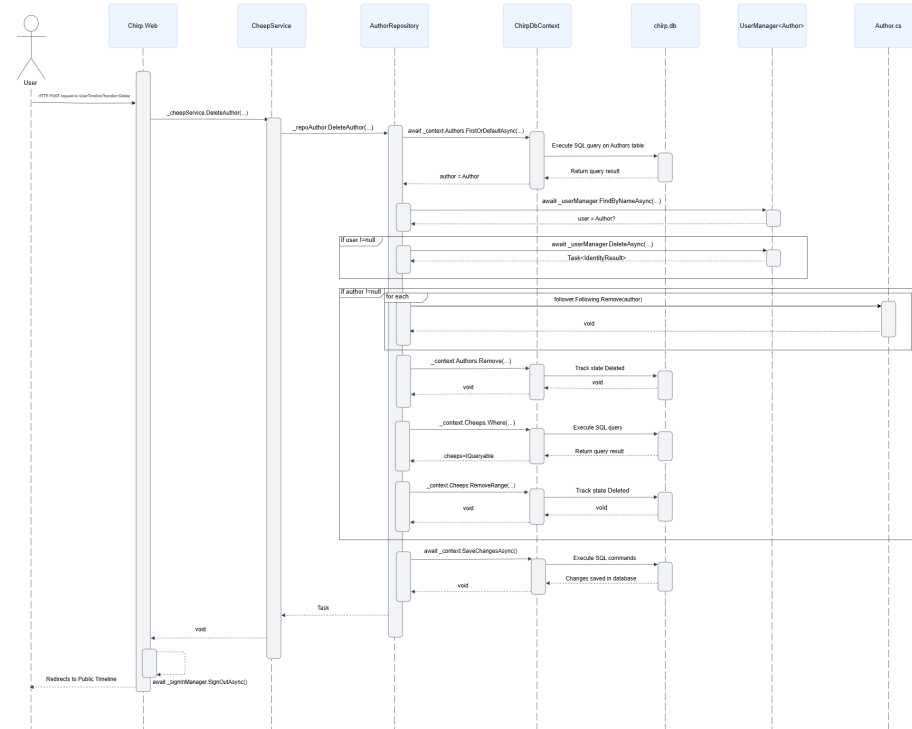
The entry point to *Chirp!* is the public timeline. The diagram shows the sequence of calls required to display the cheeps in the database to an unauthenticated user.

Follow (and unfollow)



This diagram illustrates the call sequence to follow another user. The blue and red containers represent longer functionality sequences, much like the one shown in “**Accessing the Public Timeline**” above. Unfollowing a user requires access to the same components, differing only in a few methods.

Forget me



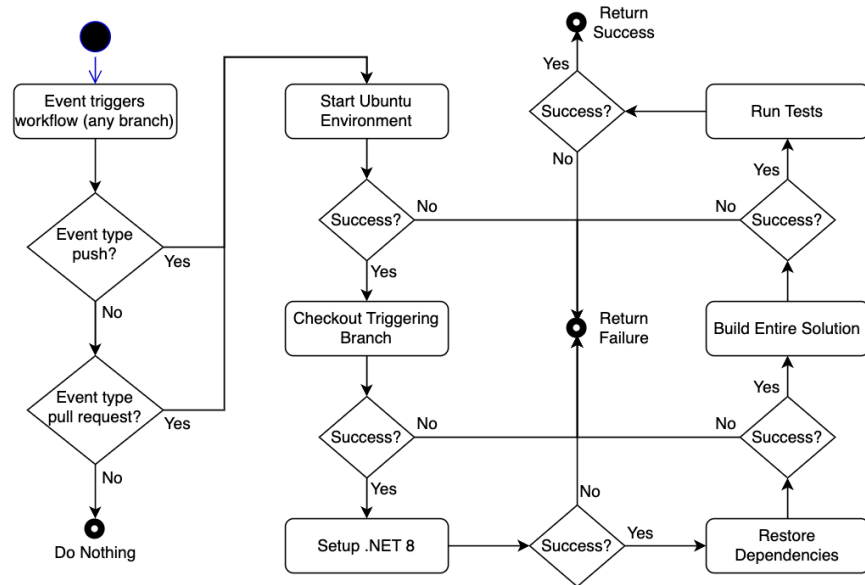
Lastly, the above diagram illustrates the sequence of calls required to delete a user from *Chirp!*. Our implementation of the “Forget Me” feature attempts to be GDPR compliant by: 1. Deleting the user from the UserManager 2. Removing the user from the followers lists of its followers 3. Removing all cheeps authored by the user 4. Executing the DELETE operation on the row in the database with the user’s information 5. Signing the user out after the above operations 6. Executing all above operations without any unnecessary delays

Process

Build, test, release, and deployment

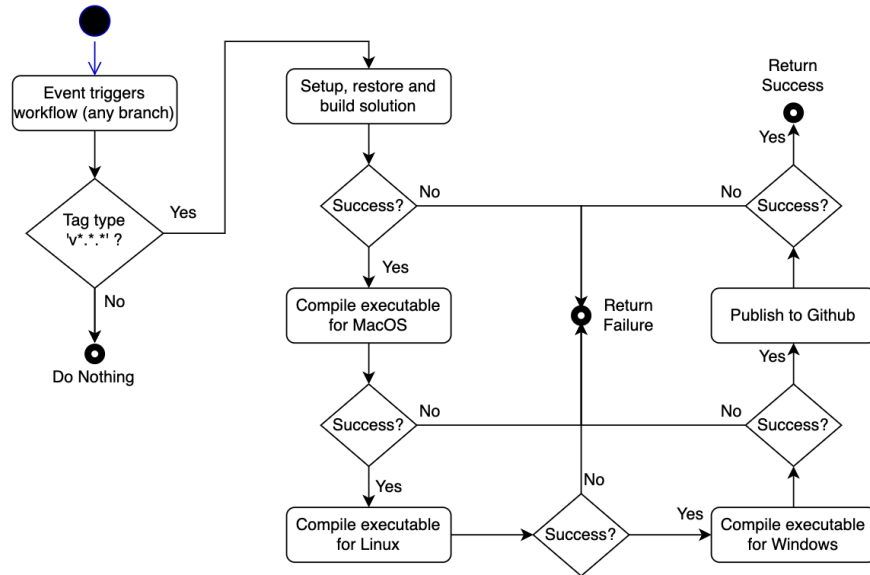
Our program is automatically built, tested, and run through the following three Github Actions Workflows:

Build and Test Workflow



This workflow shows how we automatically build and test our program on all branches whenever we push our commits or accept a pull request. This helps keep us on track with testing.

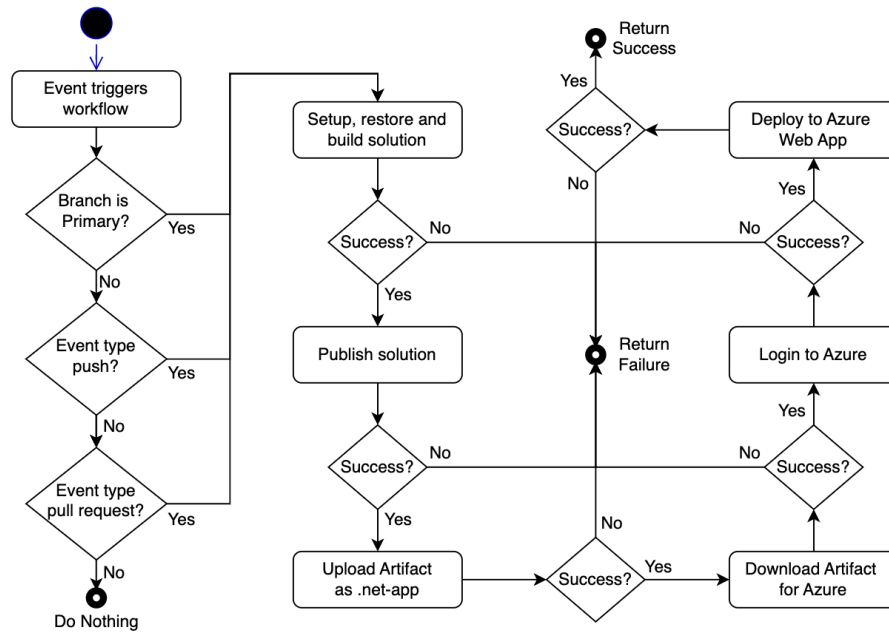
Release Workflow



This workflow shows how we automatically create releases for the three major operating systems. These releases allow a user to download and run the program locally from their own computer. The workflow is triggered by the creation and push of a new tag of the form “v..*” (Regex: the asterisks represents any number/character). The entire project is compressed into single file executables for each OS, and then further compressed together with the static files into zip files. These, together with the source code in zip and tar.gz files are released to GitHub under the version number.

The details of building the program are elaborated on in the “**Build and Test Workflow**” above.

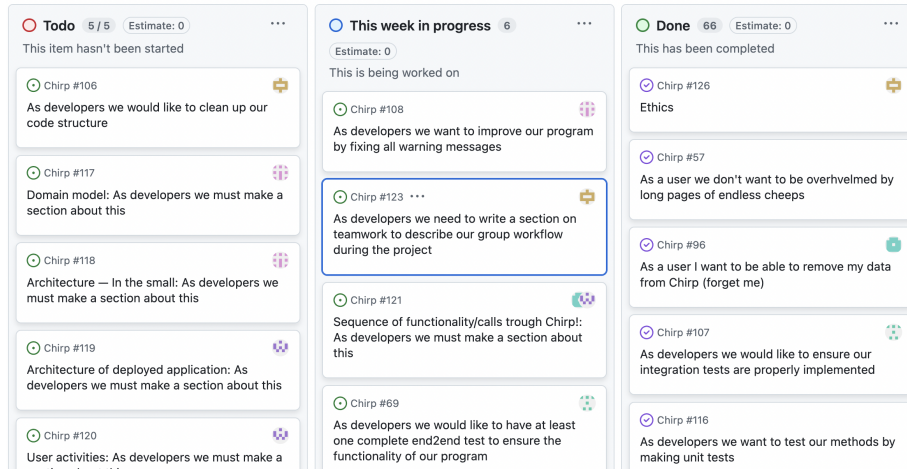
Deployment Workflow



This workflow illustrates how we deploy our program to Azure. This workflow is almost entirely built by Azure, we have simply modified it to suit the specifics of our program. The workflow is triggered by either a push or pull to the primary branch.

In this diagram too, the details of building and testing the program have been omitted, as they are elaborated on in the **“Build and Test Workflow”** above.

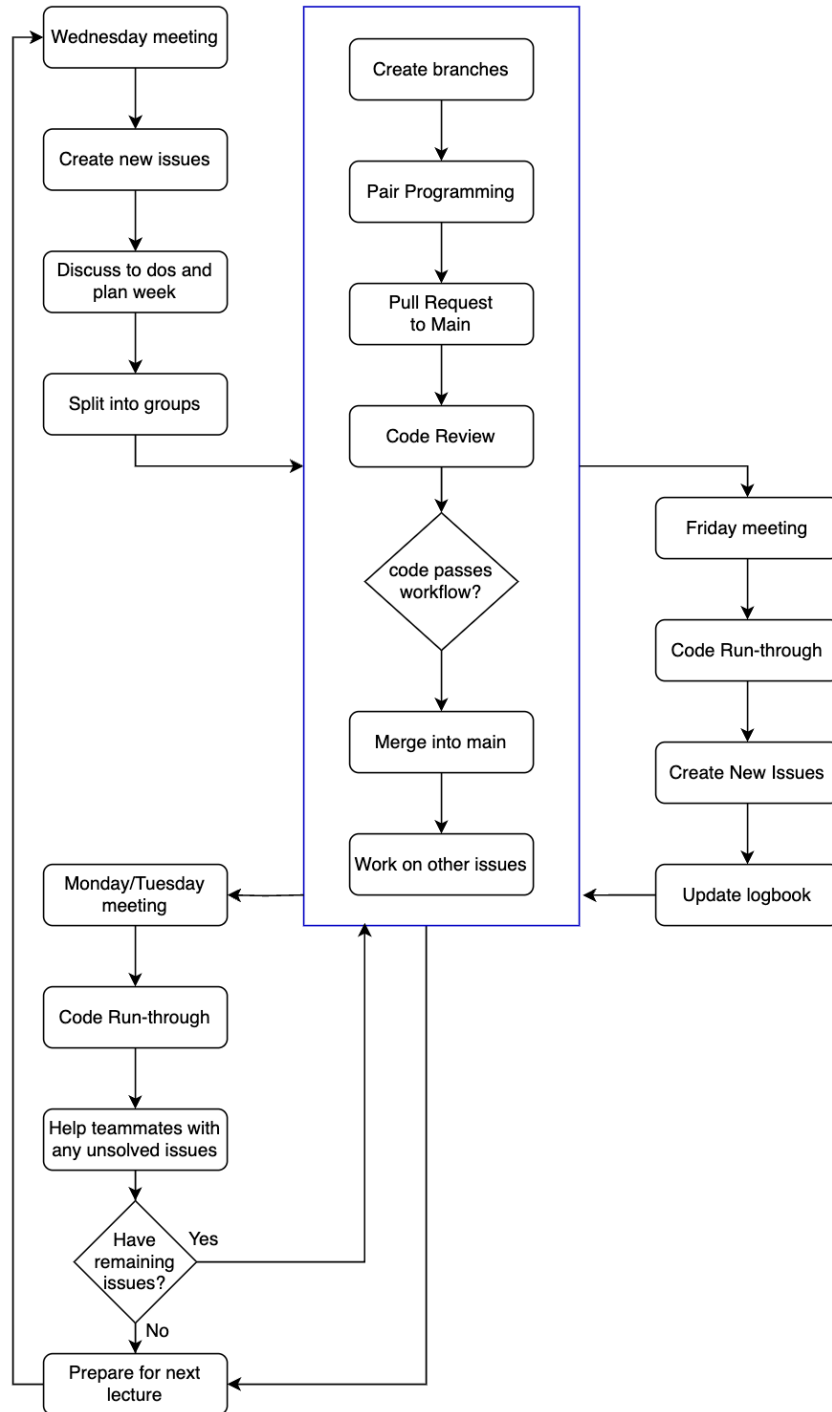
Team work



Our project board columns have been adjusted throughout this course, due to our needs varying from week to week. In the beginning of the course we were still learning to structure our time correctly, so we had columns for previous weeks that included issues we had not managed to finish before the beginning of a new week. However, with a bit of extra effort we caught up and for the last half of the course we have only had work for the current week to complete. This can be seen in the image above.

Likewise seen above, some issues have not yet been completed. This is due to us constantly improving our project these last few days, so occasionally new warnings appear, and tests need to be adjusted. These issues have therefore been ongoing for longer periods of time, and have been moved back and forth between the “This week in progress” and “Done” columns. There are also some issues on the board which reflect the status of our report at the moment of us writing this section.

Group Workflow



This is how our group tackled the weekly project work. As can be seen from the diagram, the flow in the blue box repeated throughout the week, as this is how we structured our work in smaller groups when working directly on the project.

We followed the standard pair programming strategies well throughout the weeks, and enjoyed how efficient we found this to be. As can also be seen in many of our initial commits, where all the members are credited as co-authors, we did lean towards mob programming in the beginning of the project work. This was often due to certain tasks needing to be performed sequentially, otherwise the project would not be cohesive. Additionally, we enjoyed the productive discussions that sprung from this team-working style.

We also enjoyed showing each other our work by conducting SCRUM-style code run-throughs when meeting up, as this helped us all stay up to date on the code, even the parts we had not written ourselves. This also allowed for inputs on how to improve certain parts of the code in terms of efficiency, better readability or to follow the correct architectural design models.

How to make *Chirp!* work locally

Using a release

For Windows

1. Go to our GitHub repo.
2. Click on the newest release of Chirp! found under Releases
3. Download the zip file for Windows OS
4. After the download completes, right click the zip file and extract the files.
5. After the files have been extracted, run the “Chirp.Web.exe”
6. The terminal should now show a lot of text. Find the sentence “Now listening on: http://localhost:XXXX” and note the port number.
7. Click the link or type this URL into your web browser and press enter to open the Chirp! web app.
8. You should be able to see the “Public timeline” with several cheeps displayed.
9. Press the register button in the navigation bar and register using Github or use your private email and username, and create a password.
10. You should now be able to freely explore Chirp!

For MacOS X

1. Go to our GitHub repo.
2. Click on the newest release of Chirp! found under Releases.
3. Download the zip file for MacOS.
4. After the download completes, double click the zip file and extract the folder.

5. After the folder have been extracted, right click the folder and select “New terminal at folder”.
6. Type the following command into the terminal:

```
sudo ./Chirp.Web
```

7. Enter the password for your device when prompted, and press enter.
8. If you receive the warning “‘Chirp.Web’ cannot be opened because the developer cannot be verified” follow these steps to bypass it.
 - a. Close the warning by pressing “Cancel”.
 - b. Go to “System Preferences” on your Mac.
 - c. Click on “Privacy and Security”.
 - d. Scroll to “Security”.
 - e. You should see a message saying “Chirp.Web was blocked from opening because it is not from an identified developer.”
 - f. Click on the “Allow Anyway” next to it.
 - g. Go back to the terminal and type the same command into the terminal:

```
sudo ./Chirp.Web
```

- h. Enter the password for your device when prompted, and press enter.
9. If a new warning shows up saying: “macOS cannot verify the developer of “Chirp.Web”. Are you sure you want to open it?” Press “Open”.
10. The terminal should now show a lot of text. Find the sentence “Now listening on: http://localhost:XXXX” and note the port number.
11. Click the link or type this URL into your web browser and press enter to open the Chirp! web app.
12. You should be able to see the “Public timeline” with several cheeps displayed.
13. Press the register button in the navigation bar and register using Github or use your private email and username, and create a password.
14. You should now be able to freely explore Chirp!

Using git cloning

In the **Terminal**:

1. Open a new terminal/command prompt in the folder you would like to contain Chirp, enter:

```
git clone https://github.com/ITU-BDSA23-GROUP16/Chirp.git
```


2. After the cloning process has completed, navigate into the project directory using:

```
cd Chirp
```

3. You may need to set the correct clientId and clientSecret before running. Use the following commands:

```
dotnet user-secrets init --project src/Chirp.Infrastructure
```

```
dotnet user-secrets set "authentication_github_clientId" "<clientId">  
--project src/Chirp.Infrastructure
```

```
dotnet user-secrets set "authentication_github_clientSecret" "<clientSecret">  
--project src/Chirp.Infrastructure
```

4. Run the project by entering:

```
dotnet run --project src/Chirp.Web.
```

How to run test suite locally

If you do not have playwright installed, please follow these steps first:

1. Clone the Chirp project repository (see git cloning)
2. From our Chirp repo, go to playwright testing directory

```
cd test
```

```
cd PlaywrightTests
```

3. Build the project

```
dotnet build
```

4. Add package

```
dotnet add package microsoft.playwright.MSTest
```

5. Restore

```
dotnet tool restore
```

6. Install playwright

```
dotnet playwright install
```

Then, to run test locally, follow these steps.

1. Clone the Chirp project repository (see git cloning)
2. Open your terminal
3. Find the Chirp directory using the command

```
cd <path to chirp>/Chirp/
```

4. Type in the command

`dotnet test`

5. This should run unit tests, integration tests and playwright test

Please note that the formatting of time stamps on different OS may cause the test "" to fail. Also that first time you run the tests publictimeline may timeout.

Ethics

License

For this project our group chose to use the MIT license. Due to most of the packages we use being licensed under MIT, this was the most logical choice. Moreover, we value the simplicity of the license, as we are not experienced in using licenses, and as developers we appreciate the flexibility and freedom this allows. The license gives any person “without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software”[source] under the condition that the copyright notice and the permission of MIT license is included in all copies.

[source] Our license is chosen from choosealicense.com, see file LICENSE.md

LLMs, ChatGPT, CoPilot, and others

We have used the LLM ‘ChatGPT’ a couple of times. We have made sure to mention this in our commits whenever we have done so. Each time the purpose was to gain a new perspective on a problem that had us stumped. However, it has almost always been more helpful and beneficial to ask classmates or TAs, we only resorted to the LLM when they were not available to assist. Whenever we did ask the LLM for help, it would only speed up our work approximately 50% of the time. The remaining 50% of the responses it gave to our prompts were mostly, if not entirely, useless.